9：网络编程基础

任务一：初识网络编程

首先必须要清楚：

两者处于网络通信的不同抽象层级：SocketChannel 是传输层（TCP）的字节流通道，而 HttpURLConnection 是应用层（HTTP）的协议客户端。

联系：HttpURLConnection类的底层本质也是通过SocketChannel的代码实现的，只是对Http协议进行了封装，省去了自己手写请求行、解析状态码的过程。

区别：

	HttpURLConnection	**SocketChannel**
抽象层级	应用层。进行了封装。	传输层。直接面向字节流。
协议支持	由于封装关系，仅支持HTTP/1.1	可用来承载任何协议（但要自己实现）
阻塞模型	完全阻塞	支持非阻塞（好像要配合Selector）
连接复用	有限复用	支持长连接
主要用途	自定义私有协议、需要极低延迟的场景	遗留系统维护

至于现代后端为何不再用HTTPURLConnection:

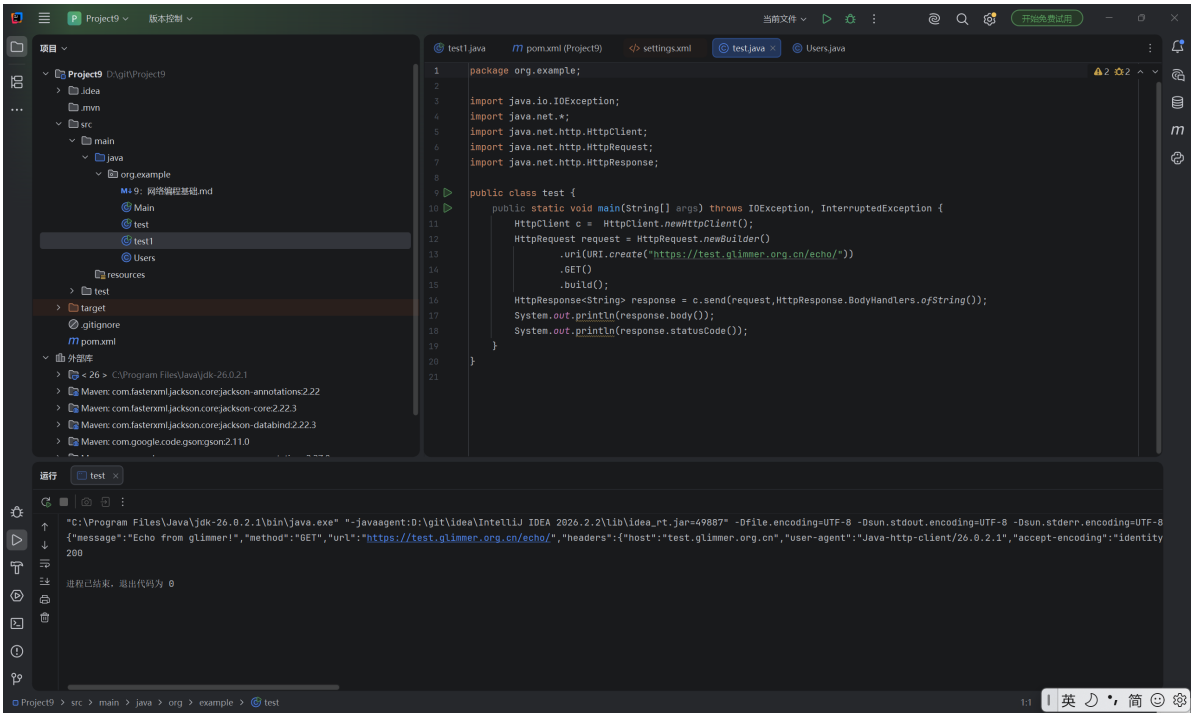
1.API设计陈旧，体验感差：具体来说，就是你连发送一个Header都要用多行代码来实现，且很容易写错。

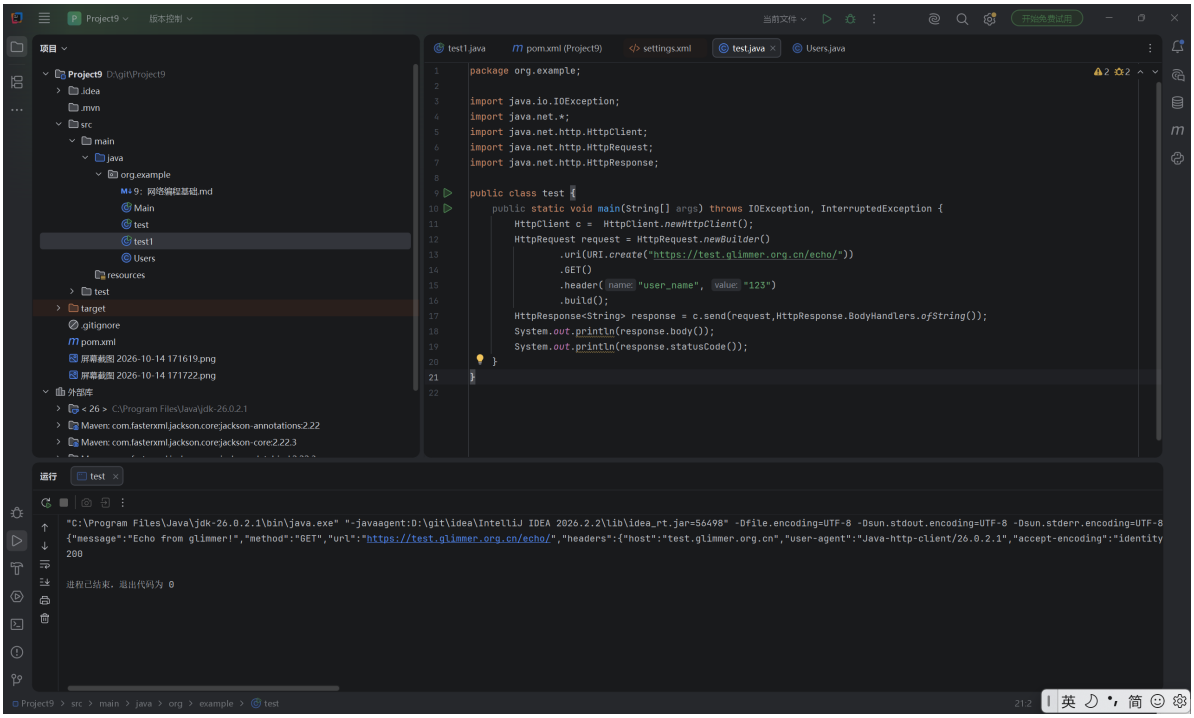
2.是一个阻塞模型：现代后端通常需要用到非阻塞的异步调用来支撑大型数据传输，而它不可能做到

任务二：上手网络编程

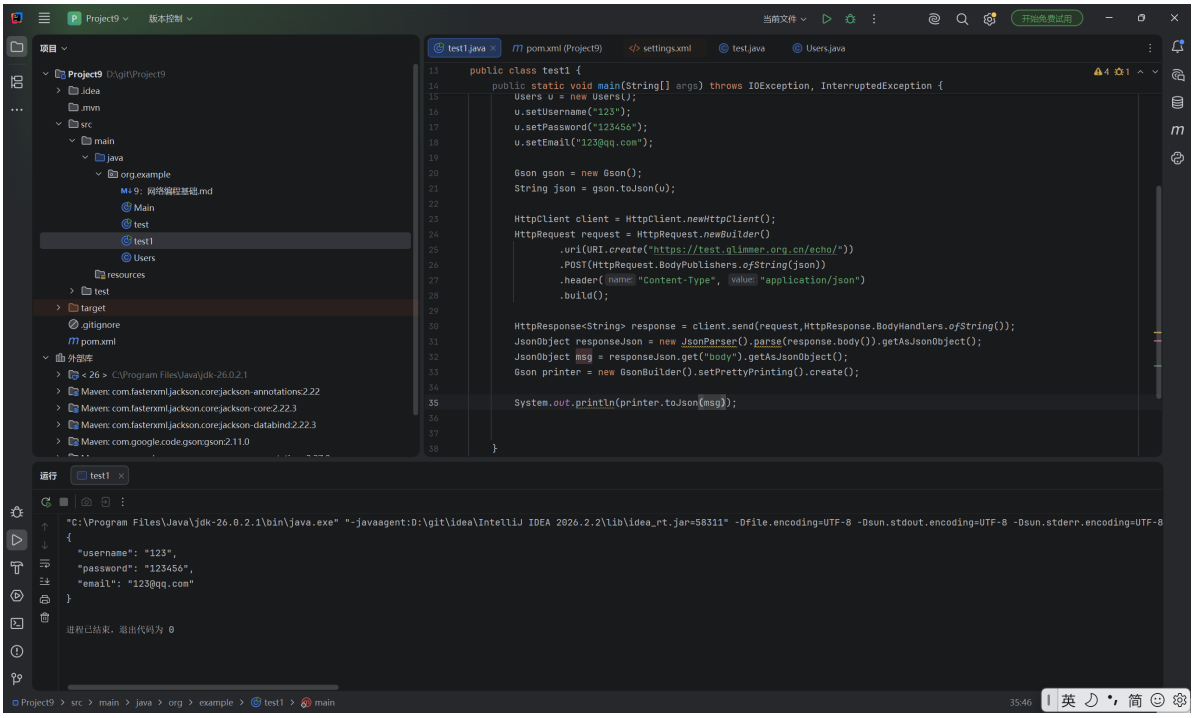
基础实践：API数据交互

不带Json:


Project9
版本控制
当前文件
开始免费试用
项目
test1.java
pom.xml (Project9)
settings.xml
test.java
Users.java
package org.example;
import java.io.IOException;
import java.net.*;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
public class test {
public static void main(String[] args) throws IOException, InterruptedException {
HttpClient c = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder()
.uri(URI.create("https://test.glimmer.org.cn/echo/"))
.GET()
.build();
HttpResponse<String> response = c.send(request,HttpResponse.BodyHandlers.ofString());
System.out.println(response.body());
System.out.println(response.statusCode());
}
}
运行
test
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe" "-javaagent:D:\git\idea\IntelliJ IDEA 2026.2.2\lib\idea_rt.jar=49887" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8
{"message":"Echo from glimmer!","method":"GET","url":"https://test.glimmer.org.cn/echo/","headers":{"host":"test.glimmer.org.cn","user-agent":"Java-http-client/26.0.2.1","accept-encoding":"identity
200
进程已结束，退出代码为 0

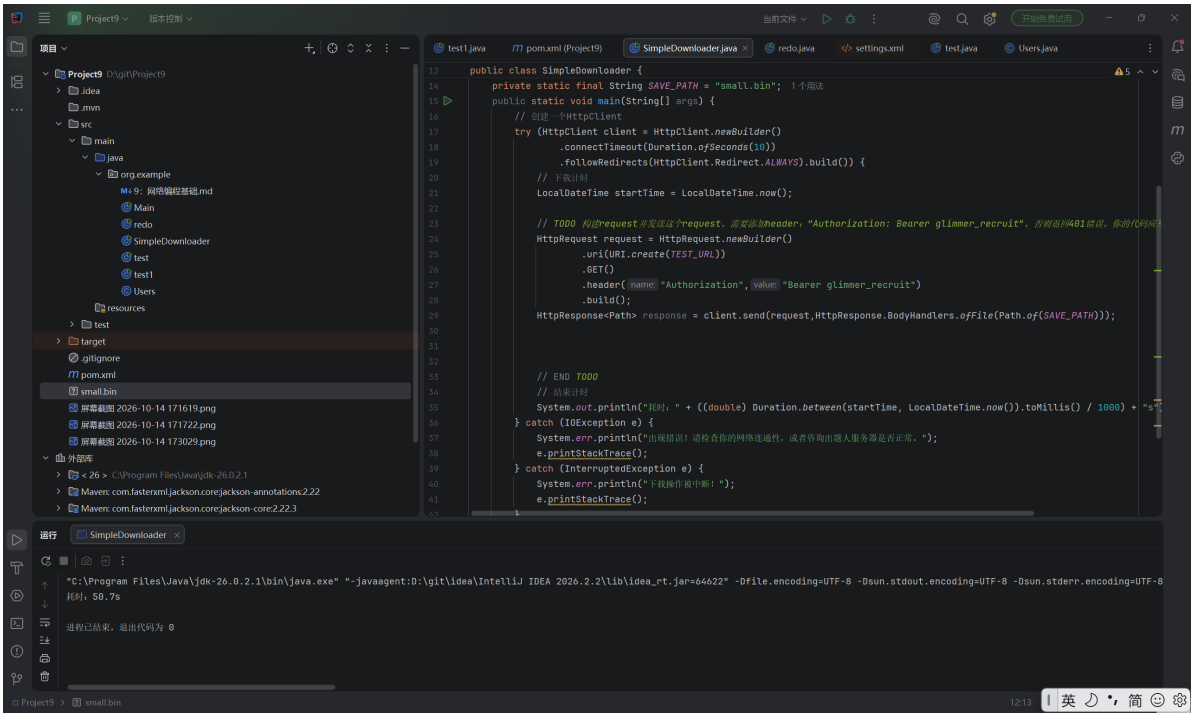

Project9
版本控制
当前文件
开始免费试用
项目
test1.java
pom.xml (Project9)
settings.xml
test.java
Users.java
package org.example;
import java.io.IOException;
import java.net.*;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
public class test {
public static void main(String[] args) throws IOException, InterruptedException {
HttpClient c = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder()
.uri(URI.create("https://test.glimmer.org.cn/echo/"))
.GET()
.header(name: "user_name", value: "123")
.build();
HttpResponse<String> response = c.send(request,HttpResponse.BodyHandlers.ofString());
System.out.println(response.body());
System.out.println(response.statusCode());
}
}
运行
test
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe" "-javaagent:D:\git\idea\IntelliJ IDEA 2026.2.2\lib\idea_rt.jar=56498" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8
{"message":"Echo from glimmer!","method":"GET","url":"https://test.glimmer.org.cn/echo/","headers":{"host":"test.glimmer.org.cn","user-agent":"Java-http-client/26.0.2.1","accept-encoding":"identity
200
进程已结束，退出代码为 0

带Json:



(搞了半天还是个回声服务器，还得把body取出来)

挑战实践：实现一个下载器



任务三：深入网络编程

查询字符串

```
HttpUrl.Builder urlBuilder = HttpUrl.prase("").newBuilder;
urlBuilder.addQuerParameter("share_source","copy_web");
urlBuilder.addQuerParameter("vd_source","3848787f4de6668e568f67f4434057ac");
String url = urlBuilder.Buil().toString;
```

责任链模式

修改后的代码如下：

```
package org.example;

import com.alibaba.fastjson2.JSON;
import okhttp3.*;

import java.io.IOException;



public class UserManager{
    public static final OkHttpClient  client1= new
OkHttpClient.Builder().addInterceptor(new Interceptor() {
        @Override
        public Response intercept(Chain chain) throws IOException {
            Request request = chain.request().newBuilder()
                    .addHeader("Authorization", auth)
                    .build();

            return chain.proceed(request);
        }

        }
    ).build();

    private static final String host = "https://test.glimmer.org.cn";
    private static final String auth = "Bearer glimmer_recruit";
    private static final OkHttpClient client = new OkHttpClient();
    public record User(long id, String nickname) {}

    // 新增用户
    public static boolean addUser(User user) {
        Request request = new Request.Builder()
                .url(host + "/users")
                .post(RequestBody.create(JSON.toJSONString(user),
MediaType.get("application/json")))
                .build();
        return execute(request);
    }

    // 删除目标id对应的用户
    public static boolean removeUser(long id) {
        Request request = new Request.Builder()
                .url(host + "/users/" + id)
```

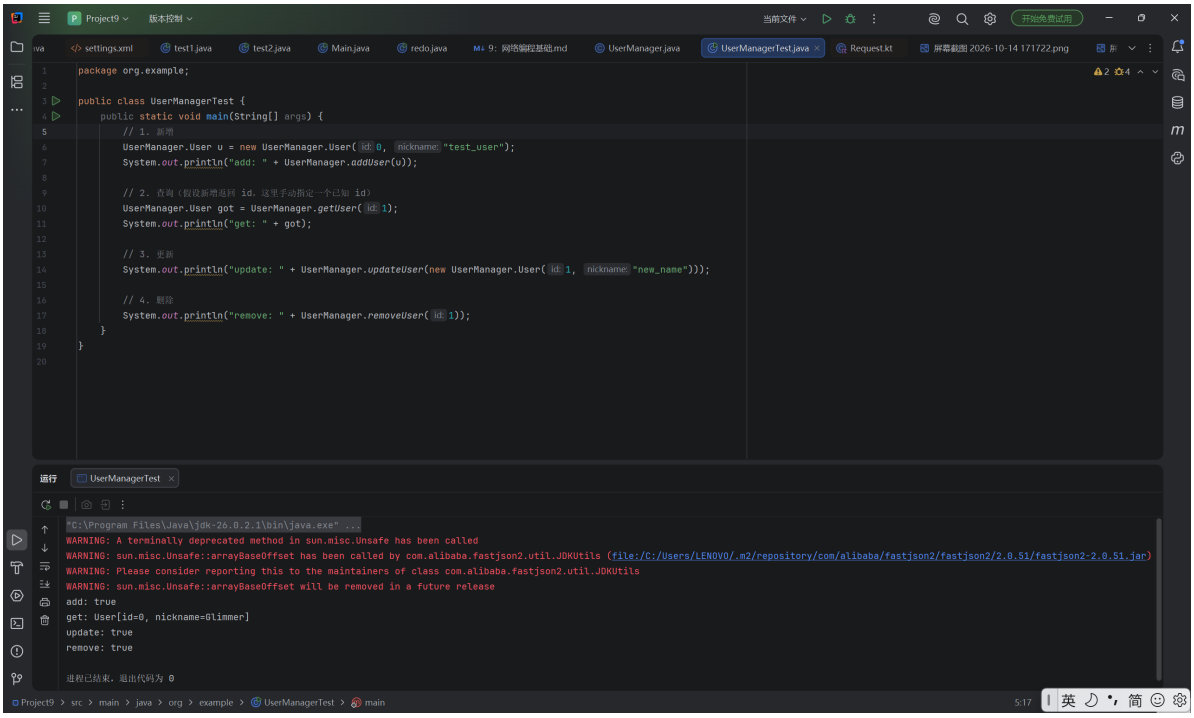
```
                .delete()
                .build();
        return execute(request);
    }

    // 更新目标用户的信息
    public static boolean updateUser(User user) {
        Request request = new Request.Builder()
                .url(host + "/users/" + user.id)
                .put(RequestBody.create(JSON.toJSONString(user),
MediaType.get("application/json")))
                .build();
        return execute(request);
    }

    // 根据id查询对应的用户
    public static User getUser(long id) {
        Request request = new Request.Builder()
                .url(host + "/users/" + id)
                .get()
                .build();
        try (Response res = client.newCall(request).execute()) {
            if (res.isSuccessful()) {
                ResponseBody body = res.body();
                if (body != null) return JSON.parseObject(body.string(),
User.class);
            }
        } catch (IOException e) {
            System.err.println("请求出现异常，请检查网络可达性");
        }
        return null;
    }

    // 简单提取重复代码，用于发送请求，并返回是否成功
    private static boolean execute(Request request) {
        try (Response res = client.newCall(request).execute()) {
            return res.isSuccessful();
        } catch (IOException e) {
            System.err.println("请求出现异常，请检查网络可达性");
        }
        return false;
    }
}
```

测试截图如下：



对于思考题，经过我的了解，okhttp中的拦截器interceptor实现的网络IO、重定向、日志、重试、自定义处理等功能拼在一起就像工厂中的流水线一样（我想到了Stream流），可以批量处理传入的请求、响应等数据，即提高了效率，又便于编程，而且具有极高的安全性。

（中间的报错好像是因为数据的安全性问题）

实践：构建你的CLI AI Agent

任务一：

